Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VIII

```
# Python3 program to print the path from root node to destination node for (N * N - 1) puzzle algorithm using Branch and Bound
# The solution assumes that instance of puzzle is solvable

# Importing copy for deepcopy function
import copy

# Importing the heap functions from python library for Priority Queue
from heapq import heappush, heappop

# This variable can be changed to change the program from 8 puzzle(n=3) to 15 puzzle(n=4) to 24 puzzle(n=5), etc.
n = 3

# Bottom, left, top, right
row = [ 1, 0, -1, 0 ]
col = [ 0, -1, 0, 1 ]
```

```
# A class for Priority Queue
class priorityQueue:

    # Constructor to initialize a Priority Queue
    def __init__(self):
        self.heap = []

    # Inserts a new key 'k'
    def push(self, k):
        heappush(self.heap, k)

    # Method to remove minimum element from Priority Queue
    def pop(self):
        return heappop(self.heap)

    # Method to know if the Queue is empty
    def empty(self):
        if not self.heap:
            return True
        else:
            return False
```

```
# Node structure
class node:

    def __init__(self, parent, mat, empty_tile_pos,
                 cost, level):

        # Stores the parent node of the current node helps in tracing path when the answer is found
        self.parent = parent

        # Stores the matrix
        self.mat = mat

        # Stores the position at which the empty space tile exists in the matrix
        self.empty_tile_pos = empty_tile_pos

        # Storesthe number of misplaced tiles
        self.cost = cost

        # Stores the number of moves so far
        self.level = level

    # This method is defined so that the priority queue is formed based on the cost variable of the objects
    def __lt__(self, nxt):
        return self.cost < nxt.cost

# Function to calculate the number of misplaced tiles ie. number of non-blank tiles not in their goal position
```

```
def calculateCost(mat, final) -> int:

    count = 0
    for i in range(n):
        for j in range(n):
            if ((mat[i][j]) and
                (mat[i][j] != final[i][j])):
                count += 1

    return count

def newNode(mat, empty_tile_pos, new_empty_tile_pos,
            level, parent, final) -> node:

    # Copy data from parent matrix to current matrix
```

```
    new_mat = copy.deepcopy(mat)

    # Move tile by 1 position
    x1 = empty_tile_pos[0]
    y1 = empty_tile_pos[1]
    x2 = new_empty_tile_pos[0]
    y2 = new_empty_tile_pos[1]
    new_mat[x1][y1], new_mat[x2][y2] = new_mat[x2][y2], new_mat[x1][y1]

    # Set number of misplaced tiles
    cost = calculateCost(new_mat, final)

    new_node = node(parent, new_mat, new_empty_tile_pos,
                    cost, level)
    return new_node

# Function to print the N x N matrix
def printMatrix(mat):

    for i in range(n):
        for j in range(n):
            print("%d " % (mat[i][j]), end = " ")

        print()

# Function to check if (x, y) is a valid matrix coordinate
def isSafe(x, y):

    return x >= 0 and x < n and y >= 0 and y < n

# Print path from root node to destination node
def printPath(root):

    if root == None:
        return

    printPath(root.parent)
    printMatrix(root.mat)
    print()

# Function to solve N*N - 1 puzzle algorithm using Branch and Bound.
# empty_tile_pos is the blank tile position in the initial state.
def solve(initial, empty_tile_pos, final):

    # Create a priority queue to store live nodes of search tree
    pq = priorityQueue()

    # Create the root node
    cost = calculateCost(initial, final)
    root = node(None, initial,
                empty_tile_pos, cost, 0)

    # Add root to list of live nodes
    pq.push(root)

    # Finds a live node with least cost, add its children to list of live nodes and finally deletes it from the list.
    while not pq.empty():

        # Find a live node with least estimated cost and delete it form the list of live nodes
        minimum = pq.pop()

        # If minimum is the answer node
        if minimum.cost == 0:

            # Print the path from root to destination
            printPath(minimum)
            return

        # Generate all possible children
        for i in range(n):
            new_tile_pos = [
                minimum.empty_tile_pos[0] + row[i],
                minimum.empty_tile_pos[1] + col[i], ]

            if isSafe(new_tile_pos[0], new_tile_pos[1]):

                # Create a child node
                child = newNode(minimum.mat,
                                minimum.empty_tile_pos,
                                new_tile_pos,
                                minimum.level + 1,
                                minimum, final,)

                # Add child to list of live nodes
                pq.push(child)
```

```
# Initial configuration
# Value 0 is used for empty space
initial = [ [ 1, 2, 3 ],
            [ 5, 6, 0 ],
            [ 7, 8, 4 ] ]

# Solvable Final configuration
# Value 0 is used for empty space
final = [ [ 1, 2, 3 ],
          [ 5, 8, 6 ],
          [ 0, 7, 4 ] ]

# Blank tile coordinates in initial configuration
empty_tile_pos = [ 1, 2 ]

# Function call to solve the puzzle
```

```
    solve(initial, empty_tile_pos, final)

    # This code is contributed by Kevin Joshi
```

```
1  2  3
5  6  0
7  8  4

1  2  3
5  0  6
7  8  4

1  2  3
5  8  6
7  0  4

1  2  3
5  8  6
0  7  4
```

Analisis Logika Pemrograman: 8-Puzzle dengan Branch and Bound

Gambaran Umum

Program ini menyelesaikan **8-puzzle** (puzzle geser 3x3) menggunakan algoritma **Branch and Bound** dengan heuristik *misplaced tiles* (jumlah ubin yang tidak pada tempatnya).

Struktur Data Utama

1. Priority Queue

```
class priorityQueue:
    def __init__(self):
        self.heap = []
    def push(self, k):
        heappush(self.heap, k)
    def pop(self):
        return heappop(self.heap)
    def empty(self):
        if not self.heap:
            return True
        else:
            return False
```

Priority Queue digunakan sebagai **open list** dalam pencarian. Node dengan nilai `cost` terendah selalu diproses terlebih dahulu. Di balik layar, `heapq` Python mengimplementasikan *min-heap* sehingga `heappop()` selalu mengembalikan elemen terkecil dalam O(log n).

---

2. Node (Simpul Pohon Pencarian)

```
class node:
    def __init__(self, parent, mat, empty_tile_pos, cost, level):
        self.parent = parent        # pointer ke node induk
        self.mat = mat              # kondisi matriks saat ini
        self.empty_tile_pos = ...   # posisi ubin kosong [baris, kolom]
        self.cost = cost            # jumlah ubin salah posisi
        self.level = level          # kedalaman / jumlah langkah

    def __lt__(self, nxt):
        return self.cost < nxt.cost
```

Setiap node merepresentasikan **satu kondisi/state** dari papan puzzle. `__lt__` didefinisikan agar objek node bisa dibandingkan oleh heap. Catatan penting: `cost` di sini hanya menyimpan nilai heuristik (jumlah misplaced tiles), **bukan** f(n) = g(n) + h(n). Artinya algoritma ini lebih mendekati *Best-First Search* daripada A*.

Fungsi-Fungsi Inti

3. calculateCost — Fungsi Heuristik

```
def calculateCost(mat, final) -> int:
    count = 0
    for i in range(n):
        for j in range(n):
            if ((mat[i][j]) and (mat[i][j] != final[i][j])):
                count += 1
    return count
```

Menghitung berapa ubin yang **tidak berada di posisi tujuan**. Ubin kosong (nilai 0) dikecualikan dari perhitungan dengan kondisi `if (mat[i][j])`. Kompleksitas: $O(n^2)$.

Contoh untuk state awal:

```
State saat ini:    State tujuan:
1  2  3            1  2  3
5  6  0            5  8  6
7  8  4            0  7  4
```

Ubin 6 ada di [1,2] padahal tujuannya [1,2] — sama, tidak dihitung. Ubin 8 ada di [2,1] padahal tujuannya [1,1] — beda, dihitung. Ubin 7 ada di [2,0] padahal tujuannya [2,1] — beda, dihitung. Ubin 0 (kosong) — dikecualikan.

Jadi `calculateCost` menghasilkan **2** untuk state awal ini.

---

4. newNode — Membuat Node Anak

```
def newNode(mat, empty_tile_pos, new_empty_tile_pos, level, parent, final):
    new_mat = copy.deepcopy(mat)
    x1, y1 = empty_tile_pos
    x2, y2 = new_empty_tile_pos
    new_mat[x1][y1], new_mat[x2][y2] = new_mat[x2][y2], new_mat[x1][y1]
    cost = calculateCost(new_mat, final)
    return node(parent, new_mat, new_empty_tile_pos, cost, level)
```

`deepcopy` dipakai agar perubahan pada matriks anak tidak mempengaruhi matriks induk. Perpindahan ubin dilakukan dengan menukar posisi ubin kosong dengan ubin tetangganya.

---

5. solve — Algoritma Utama Branch and Bound

```
def solve(initial, empty_tile_pos, final):
    pq = priorityQueue()
    cost = calculateCost(initial, final)
    root = node(None, initial, empty_tile_pos, cost, 0)
    pq.push(root)

    while not pq.empty():
        minimum = pq.pop()

        if minimum.cost == 0:
            printPath(minimum)
            return

        for i in range(n):
            new_tile_pos = [
                minimum.empty_tile_pos[0] + row[i],
                minimum.empty_tile_pos[1] + col[i]
            ]
            if isSafe(new_tile_pos[0], new_tile_pos[1]):
                child = newNode(minimum.mat, minimum.empty_tile_pos,
                                new_tile_pos, minimum.level + 1,
                                minimum, final)
                pq.push(child)
```

Alur kerja:

1. Node root dibuat dari state awal, dimasukkan ke priority queue
2. Loop utama mengambil node dengan cost terkecil
3. Jika cost == 0, semua ubin sudah di posisi tujuan, solusi ditemukan
4. Jika belum, bangkitkan hingga 4 node anak (geser bawah, kiri, atas, kanan)
5. Setiap anak yang valid dimasukkan ke priority queue
6. Proses berulang sampai solusi ditemukan

Arah gerakan dikontrol oleh:

```
row = [ 1, 0, -1, 0 ]   # delta baris:  bawah, kiri, atas, kanan
col = [ 0, -1, 0, 1 ]   # delta kolom:  bawah, kiri, atas, kanan
```

---

6. printPath — Menelusuri Jalur Solusi

```
def printPath(root):
    if root == None:
        return
    printPath(root.parent)   # rekursi ke induk dulu
    printMatrix(root.mat)    # cetak setelah kembali
    print()
```

Fungsi ini bersifat **rekursif**. Ia menelusuri pointer `parent` sampai root, kemudian mencetak matriks dari awal hingga akhir (urutan kronologis) saat stack rekursi dikembalikan.

Konfigurasi Input dan Output

```
initial = [ [ 1, 2, 3 ],
            [ 5, 6, 0 ],
            [ 7, 8, 4 ] ]

final   = [ [ 1, 2, 3 ],
            [ 5, 8, 6 ],
            [ 0, 7, 4 ] ]

empty_tile_pos = [ 1, 2 ]
```

Output yang Dihasilkan

```
1  2  3
5  6  0
7  8  4

1  2  3
5  6  4
7  8  0

1  2  3
5  6  4
7  0  8

1  2  3
5  0  4
7  6  8

1  2  3
5  8  4
7  6  0

1  2  3
5  8  0
7  6  4

1  2  3
5  8  6
7  0  4

1  2  3
5  8  6
0  7  4
```

Setiap blok matriks yang dicetak merepresentasikan satu langkah perpindahan ubin. Posisi kosong (0) bergerak dari [1,2] melewati 7 langkah hingga mencapai konfigurasi tujuan di [2,0].

Kompleksitas dan Keterbatasan

Waktu: O(b^d) di mana b adalah branching factor (maks 4) dan d adalah kedalaman solusi. Tanpa visited set, node yang sama bisa diproses berkali-kali, yang membuatnya tidak efisien untuk puzzle dengan solusi panjang.

Ruang: O(b^d) karena semua node simpan di memori melalui priority queue.

Keterbatasan utama: Program tidak memiliki mekanisme untuk menghindari state yang sudah pernah dikunjungi (*visited/closed set*). Pada puzzle dengan solusi yang panjang, ini bisa menyebabkan loop tak terbatas atau memori habis. Solusinya adalah menambahkan set berisi hash dari setiap matriks yang sudah diproses.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/